

Local AI Orchestrator

System Guide and Architecture

Private, local-first executive work orchestration on macOS
v1 scaffold — clarification implemented, execution graph in progress

Generated: 2026-05-20

Table of contents

1. What this system is
2. What this system is NOT
3. Mental model and the clarify-then-execute loop
4. Readiness scoring in depth
5. Architecture at a glance
6. Components: what each does and why
7. The safety system in depth
8. The day-unlock switch
9. Auto-execution (launchd nightly job)
10. End-to-end data flow
11. Worked example: a clarification round
12. Worked example: walking the safety gates
13. Worked example: loading and unloading a model
14. Worked example: editing day/night times
15. Sample artifacts (what a morning brief looks like)
16. Configuration recipes
17. Model selection by hardware
18. What's implemented vs scaffolded
19. File system layout
20. Database schema
21. Code roadmap (file-by-file)
22. Anti-patterns and pitfalls
23. Glossary

1. What this system is

A private, local-first AI work orchestrator running entirely on your Mac. It is **not a chatbot**. It is a system that takes vague work requests ("prepare me for tomorrow," "summarize the board notes"), turns them into structured *work packets*, asks you strong clarifying questions until the request is concrete and safe to execute, scores readiness on nine dimensions, and — eventually — executes the well-scoped work overnight using local Ollama models.

The design intent is two-fold:

- **Clarify before doing anything heavy.** Vague input never produces low-quality output; it produces questions.
- **Keep everything on your machine.** No cloud calls unless every one of six policy gates passes. No external sends in v1. No service exposure beyond localhost.

The everyday claim

If you give it a vague request during the day, it gives you back a list of questions. If you answer those questions, it scores you ready. Overnight it produces a morning brief that respects what you said. Nothing leaves your machine.

2. What this system is NOT

The negative space matters as much as the positive space. The orchestrator deliberately doesn't try to be these things:

- Not a chatbot — for conversations with a local model, use Open WebUI at <http://localhost:3000>.
- Not multi-user — single Mac, single operator, no auth.
- Not for sending anything externally — email, Slack, calendar invites, GitHub PRs, production API writes are blocked at the safety layer in v1.
- Not auto-cloud — Claude requires six gates; the day-unlock switch does not open them.
- Not running models in the background all day — DAY_MODE blocks every local-* model group unless you deliberately unlock.
- Not exposed beyond localhost — every port binds to 127.0.0.1.
- Not a knowledge base by itself — it writes to your Obsidian vault and reads from ~/LocalAI/inbox/; both are yours.
- Not auto-armed at install — the nightly launchd job is generated but never loaded without your explicit terminal command.

3. Mental model and the clarify-then-execute loop

The system is built around one loop, repeated as often as you have work. Read this table once and the rest of the architecture will follow:

Stage	What happens	Who acts
1. Capture	You give it a title and a vague description.	You
2. Clarify	The system builds an initial work packet, scores readiness on 9 dimensions, System generates up to 7 strong questions	System
3. Answer	You respond in a markdown file or via the panel.	You
4. Re-score	The system parses your answers, rescores, and either marks the packet READY_FOR_OVERNIGHT (≥ 0.85)	System
5. Execute	Overnight, in NIGHT_MODE, the execution graph reads sources, calls local System (scaffolding) safety gate, evaluates	System (scaffolding)
6. Review	You read the output in ~/LocalAI/output/<date>/ and the Obsidian vault; approve memory candidates for the long	You

Two key properties of the loop:

- **The system never executes without your consent.** Readiness threshold is the gate; below 0.85 (or 0.90 for high-stakes) the work refuses to start.
- **The clarification step is valuable by itself.** Most weekly value comes from being forced to be explicit about quality, audience, sources, and escalation — even when overnight execution never runs.

4. Readiness scoring in depth

Readiness is a deterministic weighted sum over nine binary dimensions. Each dimension is 1 if present in the packet, 0 if missing. The score lands in [0, 1].

Dimension	Weight	What it captures	Blocking?
objective	0.18	Is the desired outcome explicit?	yes
output_format	0.14	Is the output shape specified?	yes
sources	0.14	Which folders/files are in scope?	yes
privacy_cloud_policy	0.14	Is cloud explicitly allowed or forbidden?	yes
quality_threshold	0.12	What quality bar to optimize for?	no
audience	0.10	Who is the output for?	no
assumption_policy	0.08	May the system infer when uncertain?	no
escalation_policy	0.06	When should it stop and ask?	no
success_criteria	0.04	What makes the work done?	no

Thresholds and status mapping

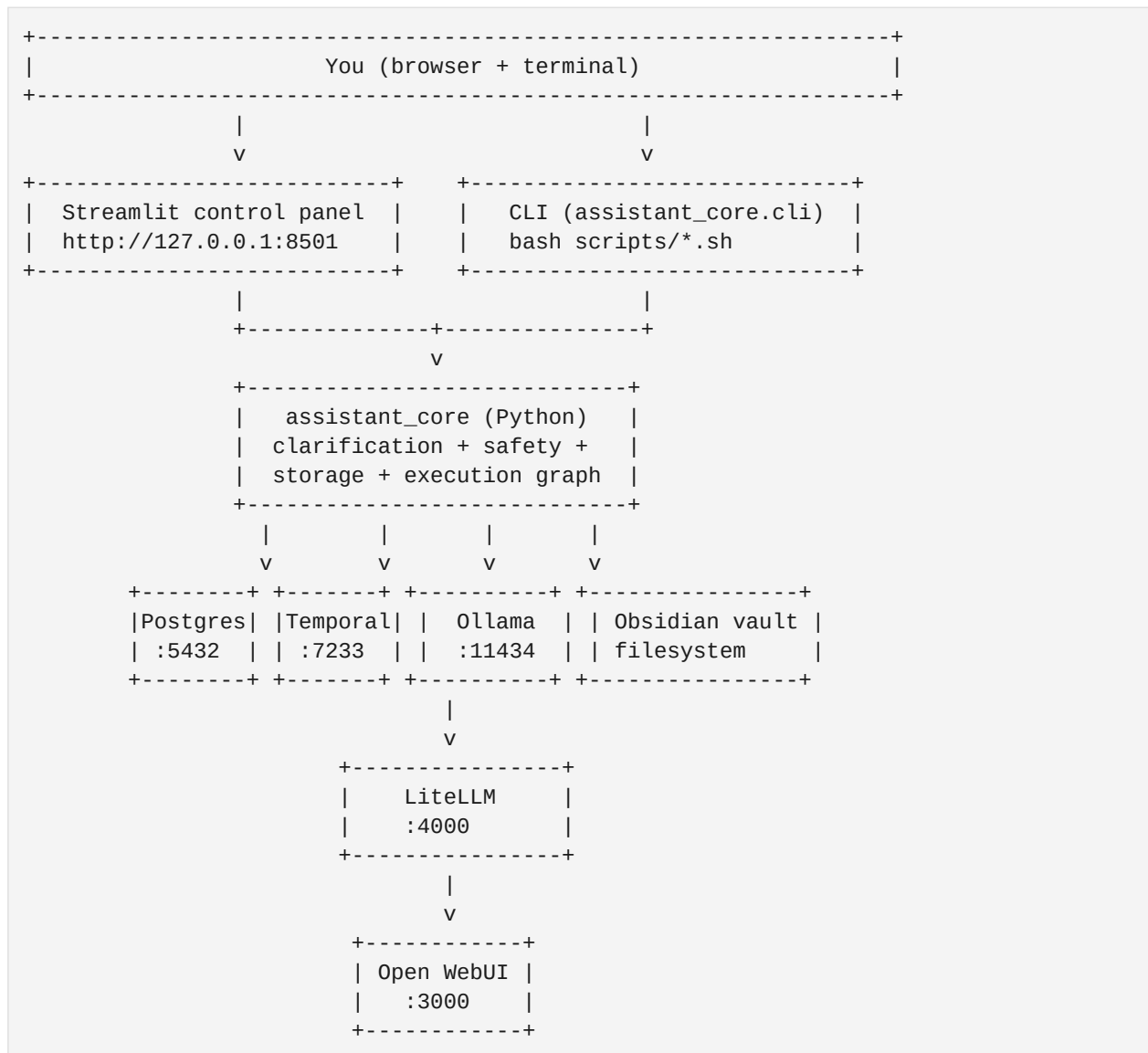
Score range	Status	Eligible for overnight?
>= 0.90	READY_HIGH_CONFIDENCE	Yes (required for high-stakes)
0.85 to 0.89	READY_FOR_OVERNIGHT	Yes for normal packets; no for high-stakes
0.65 to 0.84	NEEDS_CLARIFICATION	No; another question round
< 0.65	UNDERDEFINED	No; major gaps remain

High-stakes auto-detection: the work packet builder scans title and description for any of: board, CEO, legal, investor, high-stakes. If found, `high_stakes=True` is set and the threshold becomes 0.90.

Blocking vs non-blocking gaps: the four blocking dimensions (objective, output_format, sources, privacy_cloud_policy) are weighted heavily and must all be 1 to even reach 0.85. The other five contribute to fine-tuning but cannot individually keep a packet from going ready.

5. Architecture at a glance

Nine components, all running locally, talking to each other on 127.0.0.1.



Solid lines: required. LiteLLM and Open WebUI are optional in v1 — the panel talks to Ollama directly through `assistant_core.llm.ollama_admin`.

What runs where

Component	Lifecycle	Started by
Streamlit panel	Host Python process	bash scripts/start_control_panel.sh
assistant_core CLI	Host Python process (one-shot)	bash scripts/*.sh
Postgres	Docker container	bash scripts/start_services.sh
Temporal + Temporal UI	Docker containers	bash scripts/start_services.sh

Component	Lifecycle	Started by
Open WebUI	Docker container	bash scripts/start_services.sh
Ollama	Host service (brew)	brew services start ollama
LiteLLM	Host Python process	bash scripts/start_litellm.sh
Temporal worker	Host Python process (long-running)	bash scripts/start_temporal_worker.sh

Important consequence: bash scripts/stop_services.sh only stops the Docker containers (Postgres, Temporal, Temporal UI, Open WebUI). It does not touch Streamlit, Ollama, or LiteLLM — those are host processes and must be stopped separately (Ctrl+C their terminals, or kill \$(lsof -nP -iTCP:<port> -sTCP:LISTEN -t)).

6. Components: what each does and why

Python orchestrator (assistant_core)

The brain. Pure Python, no framework.

What it does

Holds the deterministic clarification logic, the safety gates, the work-packet schema, the storage layer, and the (scaffolded) execution graph. Has no opinion about UI — both the Streamlit panel and the CLI call into it. Tested in isolation via pytest.

Why this tool

Pure Python keeps the core auditable, easy to reason about, easy to test. Pydantic for schemas (validation at type boundaries). YAML for human-editable config. psycopg for direct Postgres I/O without an ORM layer that hides intent.

Where to look

```
assistant_core/{clarification,llm,memory,execution,temporal_app}/
```

Streamlit control panel

Local web UI on :8501.

What it does

Renders the status strip (mode, day unlock, services up/down), seven tabs (Dashboard, Create Work Packet, Clarification, Models, Schedule, Execution, Artifacts), and is the only UI surface for safe model load/unload.

Why this tool

Streamlit lets us iterate in pure Python without writing a frontend. Tabs are stateless re-renders — safe by construction. Editing config or invoking Ollama always goes through assistant_core, never bypasses the gate.

Where to look

```
app/control_panel.py
```

PostgreSQL

Operational state store.

What it does

Holds work_packets, clarification_questions, execution_runs, model_calls, evaluations, artifacts, memory_candidates. Migrated automatically on first container start (db/migrations/001_init.sql is mounted into Postgres' init dir).

Why this tool

Postgres because we want durable, queryable state with relational integrity across the seven tables — work packets relate to questions, runs, model calls, and evaluations. JSON columns for cloud/source/output policy let us evolve without migrations.

Where to look

```
db/migrations/001_init.sql + assistant_core/storage/
```

Temporal

Durable workflow engine on :7233 (UI on :8233).

What it does

Manages overnight workflows: clarification, execution, manual resume. Survives process crashes; can pause and resume long-running work. The worker process (scripts/start_temporal_worker.sh) executes activities defined in assistant_core/temporal_app/.

Why this tool

Overnight work runs for hours, may hit transient model failures, and needs to be resumable. Temporal handles retries, timeouts, idempotency, and audit trail out of the box. Cron + bash would be fragile at this scale.

Where to look

```
assistant_core/temporal_app/{workflows,activities,worker,client}.py
```

Ollama

Local model runtime on :11434.

What it does

Pulls models to ~/.ollama/models on disk, loads them into Metal VRAM on demand, serves inference via HTTP. The orchestrator never bypasses Ollama for local model calls — it always goes through this daemon.

Why this tool

Ollama is the de facto local-model runtime on macOS Metal. Native Apple Silicon support, good keep-alive semantics, simple HTTP API, library of pre-quantized models. Lighter and easier to operate than running llama.cpp directly.

Where to look

```
config/models.yaml maps groups -> Ollama tags
```

LiteLLM

Model router and gateway on :4000.

What it does

Translates a model-group name (local-main, local-coder, etc.) into the right Ollama tag or cloud endpoint. Configured with local-only fallbacks; cloud is intentionally not chained behind local.

Why this tool

LiteLLM gives us a single OpenAI-style API for code that wants to swap models later, with fallback rules expressed declaratively. We keep the cloud route in the config but disable cloud at the safety layer so no blind escalation can happen.

Where to look

```
config/litellm.yaml.example + assistant_core/llm/litellm_client.py
```

Open WebUI

ChatGPT-style chat against local models, on :3000.

What it does

A separate tool that lives next to the orchestrator. Talks directly to Ollama. Provides conversational access when you want it; the orchestrator deliberately does not offer chat.

Why this tool

Sometimes you want to chat with a model. Open WebUI is a polished, mature tool for that. Co-installing it via docker-compose lets you have both surfaces.

Where to look

```
docker-compose.yml — service: open-webui
```

Obsidian vault

Human-readable long-term notes.

What it does

The system writes outputs and memory candidates into `~/Obsidian/LocalAI-ChiefOfStaff/`. Daily briefs, work packets, decisions, meeting notes, memory review queue. You read and curate these directly in Obsidian.

Why this tool

Obsidian's plaintext markdown vault is durable, indexable, and yours. No lock-in. The orchestrator writes; you review. The vault becomes your long-term knowledge layer.

Where to look

```
~/Obsidian/LocalAI-ChiefOfStaff/{01_Daily_Briefs, 02_Work_Packets, ...}
```

Chroma (vector store)

Local semantic retrieval target.

What it does

Scaffolded but not yet wired. Will hold embeddings of selected outputs and vault content for retrieval-augmented work.

Why this tool

Chroma is simple, file-backed, no separate daemon. Right tradeoff for a single-user local system. Persistent client at `~/LocalAI/state/chroma`.

Where to look

```
assistant_core/memory/vector_store.py
```

Docker Compose

Service orchestration for Postgres / Temporal / Open WebUI.

What it does

One file, four services. Containers expose ports only on 127.0.0.1. Named volumes persist data across container recreates. The migration SQL is mounted into Postgres' init directory so the schema is created on first run.

Why this tool

Docker keeps these three services consistent across machines without ceremony. They're stateful, multi-process, and have non-trivial init — containers handle that cleanly.

Where to look

```
docker-compose.yml
```

7. The safety system in depth

The orchestrator is built around the assumption that you do not want the system to make decisions you wouldn't make yourself. Every model call, every cloud fallback, every external write passes through a deterministic gate in `assistant_core/safety.py`. Gates are written as *fail-closed* — an unknown tag, an empty env var, or a missing flag all default to **blocked**.

The five gate functions

Function	What it gates
<code>assert_cloud_allowed</code>	Six checks before any Claude call: <code>cloud_fallback_enabled</code> , <code>work_packet_cloud_allowed</code> ,
<code>assert_no_external_write</code>	Always blocks in v1. Used at any future email/Slack/API boundary.
<code>assert_original_file_write_allowed</code>	Refuses writes outside the configured roots (output, archive, logs, work_packets, vault).
<code>assert_heavy_execution_allowed</code>	Refuses model execution in <code>DAY_MODE</code> unless day unlock is active or <code>manual_override=</code>
<code>assert_model_allowed</code>	Top-level entry point. Rejects cloud-claude-opus outright; then delegates local groups to a

Modes

LOCALAI_MODE value	Meaning	Local model calls allowed?
<code>DAY_MODE</code> (default)	Workday: clarification + deterministic logic only.	No, unless <code>day_unlock</code> active
<code>NIGHT_MODE</code>	Overnight: heavy execution permitted.	Yes
<code>MANUAL_RESUME</code>	User-driven re-run of paused work.	Yes

Mode is a single env var. The default is `DAY_MODE`. The overnight runner refuses to start unless `NIGHT_MODE` is set; the `launchd` plist sets it automatically when it fires.

8. The day-unlock switch

DAY_MODE blocks all four local-* model groups (local-main, local-secondary, local-coder, local-reasoner). To deliberately work with models during the day, the day-unlock switch flips a single boolean. Two channels:

- **Sentinel file** — `~/LocalAI/state/day_unlock.flag`. Created by `scripts/day_unlock.sh` with a timestamp, hostname, and reason. Persists across terminal closes and reboots.
- **Environment variable** — `LOCALAI_DAY_UNLOCK=true`. Per-shell, ephemeral.

Either being on opens the gate. Cloud and external-write gates are **not** affected. The Streamlit panel renders a red banner with the flag contents while unlock is ACTIVE so the state is impossible to miss.

Sentinel file format

```
$ cat ~/LocalAI/state/day_unlock.flag
unlocked_at: 2026-05-17T03:48:22Z
unlocked_by: user@host
reason: spot-checking the morning brief
```

Contents are informational only — the safety layer never parses them. An empty touch `~/LocalAI/state/day_unlock.flag` also unlocks. The structure is for you (or for a future you reviewing why the flag was left on).

Full design rationale: [docs/day_unlock.md](#). Locking again: `bash scripts/day_lock.sh`.

9. Auto-execution (launchd nightly job)

Default state: **OFF**. The orchestrator can be scheduled to run overnight via macOS launchd, but the plist is never armed automatically. Arming requires three deliberate steps in the terminal.

The four states

Project plist	Installed in LaunchAgents	Loaded in launchctl	State
no	no	no	fully off, nothing installed
yes	no	no	off (default after install_core)
yes	yes	no	off, installed but not loaded
yes	yes	yes	ARMED, will run nightly at night_mode_start

Arming (three deliberate steps)

```
# 1. Verify the configured time is what you want.
#   Edit it in the panel's Schedule tab or in config/assistant.yaml.

# 2. Regenerate the plist with that time:
bash scripts/create_launchd_plist.sh

# 3. Install into LaunchAgents and load:
LOCALAI_WRITE_LAUNCHD=true bash scripts/create_launchd_plist.sh
launchctl load ~/Library/LaunchAgents/com.localai.orchestrator.nightly.plist
```

Disarming

```
launchctl unload ~/Library/LaunchAgents/com.localai.orchestrator.nightly.plist
# Optionally also delete the installed copy:
rm ~/Library/LaunchAgents/com.localai.orchestrator.nightly.plist
```

How to verify the state from anywhere

```
launchctl list | grep com.localai.orchestrator.nightly
ls ~/Library/LaunchAgents/com.localai.orchestrator.nightly.plist
ls launchd/com.localai.orchestrator.nightly.plist
crontab -l | grep -i localai
```

Full design rationale: docs/auto_execution.md.

10. End-to-end data flow

From a vague request to overnight output, in nine steps. The next section walks the same flow with concrete JSON.

Step	What happens	Storage
1	You: bash scripts/create_work_packet.sh "Title" "Description"	—
2	build_initial_work_packet builds a WorkPacket pydantic model.	in memory
3	score_readiness assigns 0/1 per dimension, computes weighted score	in memory
4	generate_questions emits up to 7 ranked clarification questions.	in memory
5	create_work_packet + save_questions write the row and questions to postgres	Postgres work_packets, clarification_questions
6	You answer in a markdown file. bash scripts/answer_questions.sh &path;	&path;
7	update_packet_from_answers extracts fields. Rescore. If READY, packages up	Postgres update work_packets
8	Overnight runner (manual or via launchd at NIGHT_MODE) finds READY packets, submits	Postgres packets, submits Temporal workflows.
9	Workflow body (scaffolded) reads sources, calls models, evaluates, writes output	Files, system output, Postgres artifacts, model calls, and Data

Step 9 is still scaffolded as of this writing. The workflow is wired and the Temporal worker accepts the submission, but the activity body returns a placeholder dict instead of running the 18-step pipeline. That is the next feature-work chunk.

11. Worked example: a clarification round

A complete walk through a real clarification. The same call you would make from the CLI, with the exact JSON the system returns at each step.

Step 1: vague request

```
bash scripts/create_work_packet.sh \  
  "Morning prep" \  
  "Prepare me for tomorrow from my notes."
```

Step 2: response — rejected for clarification

```
{  
  "work_packet_id": "f37b8a92-...-4c11",  
  "status": "UNDERDEFINED",  
  "readiness_score": 0.18,  
  "questions": [  
    {  
      "category": "objective",  
      "question": "What exact outcome should this work produce, and what should be c  
onsidered out of scope?",  
      "priority": 100, "blocking": true  
    },  
    {  
      "category": "outputs",  
      "question": "What exact output files or formats do you want at the end?",  
      "priority": 95, "blocking": true  
    },  
    {  
      "category": "sources",  
      "question": "Which folders or files are in scope, and are any sources explicit  
ly off-limits?",  
      "priority": 90, "blocking": true  
    },  
    {  
      "category": "privacy/cloud",  
      "question": "Is cloud fallback explicitly allowed for this packet, or must it  
stay fully local?",  
      "priority": 88, "blocking": true  
    },  
    ...  
  ]  
}
```

Score 0.18 is below 0.65, so status is UNDERDEFINED. Four blocking dimensions are missing (objective is too generic to satisfy the check; the rest have nothing). The system refuses to proceed.

Step 3: answer markdown

```
# examples/sample_answers.md  
  
Outputs: morning brief, today's priorities, risks and blockers, meeting prep.
```

Sources: only ~/LocalAI/inbox/notes and ~/LocalAI/inbox/docs.
Cloud: do not use cloud fallback.
Audience: private user only.
Assumptions: infer low-risk priorities but label every assumption.
Quality: optimize for factuality, criticality, and actionability.
Stop conditions: stop for missing sources, unclear ownership, or anything requiring external action.
Success: I can read it in the morning and know what matters first.

Step 4: submit answers

```
bash scripts/answer_questions.sh \  
  f37b8a92-...-4c11 \  
  examples/sample_answers.md
```

Step 5: response — ready

```
{  
  "work_packet_id": "f37b8a92-...-4c11",  
  "status": "READY_FOR_OVERNIGHT",  
  "readiness_score": 0.9,  
  "database": "updated",  
  "remaining_questions": []  
}
```

All four blocking dimensions are filled (objective is the request itself, outputs and sources extracted from the markdown, cloud policy explicit). The five non-blocking dimensions add up to 0.40 / 0.40 (audience, quality, assumptions, escalation, success all answered). Score lands at 0.90 = READY_HIGH_CONFIDENCE.

What the packet looks like in Postgres now

```
SELECT id, status, readiness_score, high_stakes  
FROM work_packets  
WHERE id = 'f37b8a92-...-4c11';
```

id	status	readiness_score	high_stakes
f37b8a92-...-4c11	READY_FOR_OVERNIGHT	0.90	false

12. Worked example: walking the safety gates

A Python REPL session showing each gate in turn. Useful for understanding what the gate functions actually do without reading the code.

DAY_MODE blocks local-main by default

```
$ .venv/bin/python
>>> from assistant_core.safety import assert_model_allowed, SafetyError
>>> try:
...     assert_model_allowed('local-main', 'DAY_MODE', manual_override=False)
... except SafetyError as e:
...     print('blocked:', e)
blocked: Local model execution is blocked in DAY_MODE without explicit
manual override.
```

Manual override unblocks it

```
>>> assert_model_allowed('local-main', 'DAY_MODE', manual_override=True)
>>># returns None silently &mdash; call would proceed
```

Day-unlock flag does the same thing without per-call override

```
$ bash scripts/day_unlock.sh "REPL demo"
>>> assert_model_allowed('local-main', 'DAY_MODE', manual_override=False)
>>># returns None silently
$ bash scripts/day_lock.sh
>>> assert_model_allowed('local-main', 'DAY_MODE', manual_override=False)
SafetyError: Local model execution is blocked in DAY_MODE without explicit
manual override.
```

Cloud Claude is always blocked here

```
>>> assert_model_allowed('cloud-claude-opus', 'NIGHT_MODE', manual_override=True)
SafetyError: Claude must be authorized through cloud policy checks, not
model selection alone.
```

Cloud Claude has its own dedicated gate. Even with NIGHT_MODE + manual_override + day unlock all set, you cannot reach Claude through this function. The right path is `assert_cloud_allowed(...)` which checks six independent conditions.

Cloud Claude with the proper gate (still mostly fails)

```
>>> from assistant_core.safety import assert_cloud_allowed
>>> assert_cloud_allowed(
...     file_path='~/LocalAI/inbox/cloud_review/brief.cloud.md',
...     cloud_fallback_enabled=False, # config gate -> raises
...     work_packet_cloud_allowed=True,
...     high_stakes=True,
...     local_quality_gate_failed=True,
...     anthropic_api_key='sk-...',
...     daily_budget_remaining=True,
...     cloud_review_dir='~/LocalAI/inbox/cloud_review',
```

```
... )  
SafetyError: Cloud fallback is disabled by configuration.
```

Six different gate conditions; flipping any one false raises. To actually reach Claude you need to satisfy all six simultaneously. That's deliberate.

13. Worked example: loading and unloading a model

The Models tab and the underlying API calls. Same effect from either entry point.

From the panel

- Browse to `http://127.0.0.1:8501` -> **Models** tab.
- Day unlock must be ACTIVE if you're in DAY_MODE — run `bash scripts/day_unlock.sh "reason"` first if needed.
- Click **Load** next to `deepseek-r1:8b`. Spinner. Success toast.
- Model now appears in *Currently loaded* with size, expires_at, and Unload button.
- Scroll to **Quick test prompt**, pick the model, type a prompt, click Send.
- Click **Unload**. Model disappears from *Currently loaded*.

What the panel does behind the scenes (curl equivalents)

```
# Load with 30-minute keep-alive
curl -X POST http://localhost:11434/api/generate \
  -d '{"model":"deepseek-r1:8b","prompt":"","keep_alive":"30m"}'

# What is currently in VRAM
curl http://localhost:11434/api/ps

# Send a prompt
curl -X POST http://localhost:11434/api/generate \
  -d '{"model":"deepseek-r1:8b","prompt":"Say hi.,"stream":false}'

# Unload (note the string '0s', not int 0 &mdash; some Ollama versions parse
# int 0 as "use default 5m" and silently re-pin the model)
curl -X POST http://localhost:11434/api/generate \
  -d '{"model":"deepseek-r1:8b","prompt":"","keep_alive":"0s"}'
```

Equivalent CLI

```
ollama list          # what's on disk
ollama ps            # what's in VRAM
ollama run deepseek-r1:8b  # interactive chat (not what the orchestrator uses)
ollama stop deepseek-r1:8b # immediate unload (sends keep_alive=0s)
```

What the gate sees

```
# Inside ollama_admin.load_model('deepseek-r1:8b', mode='DAY_MODE'):
1. resolve_group_for_tag('deepseek-r1:8b') -> 'local-reasoner'
2. assert_model_group_allowed('local-reasoner', 'DAY_MODE')
   -> calls assert_model_allowed
       -> not cloud-claude-opus, so skip first check
       -> 'local-reasoner' in LOCAL_MODEL_GROUPS, delegate
```

```
-> assert_heavy_execution_allowed('DAY_MODE')
-> day_unlock_active() reads ~/LocalAI/state/day_unlock.flag
-> if ACTIVE: pass; if not: raise SafetyError
3. If gate passes: POST /api/generate with keep_alive='30m'
```

14. Worked example: editing day/night times

Say you want the nightly job to start at 02:00 instead of 01:30, and treat 06:30 as the end of night. Two ways: edit the config file directly, or use the panel.

Via the panel

- Browse to the panel -> **Schedule** tab.
- Edit `night_mode_start` to 02:00 and `night_mode_end` to 06:30.
- Click **Save to config/assistant.yaml**. Atomic write; validation runs first.
- If the plist is already installed in LaunchAgents, a yellow warning appears with the `regenerate+reload` commands.

Via the terminal

```
# Edit config/assistant.yaml by hand or with sed; the panel and the safety
# layer pick it up automatically on next read.

# Then if the plist is installed:
LOCALAI_WRITE_LAUNCHD=true bash scripts/create_launchd_plist.sh
launchctl unload ~/Library/LaunchAgents/com.localai.orchestrator.nightly.plist
launchctl load ~/Library/LaunchAgents/com.localai.orchestrator.nightly.plist

# Verify the new time is baked into the installed plist:
grep -A3 StartCalendarInterval \
  ~/Library/LaunchAgents/com.localai.orchestrator.nightly.plist
```

What does NOT change automatically

- `LOCALAI_MODE` stays at whatever the shell sets it to. Editing the times does not auto-flip mode.
- The launchd plist installed in `~/Library/LaunchAgents` is not regenerated until you run the script with `LOCALAI_WRITE_LAUNCHD=true`.
- Already-running services don't restart.
- Already-running Temporal workflows continue with whatever mode they were started in.

15. Sample artifacts (what a morning brief looks like)

When the overnight pipeline is fully wired, each run produces a dated directory under `~/LocalAI/output/YYYY-MM-DD/` containing the following nine files. The shapes are committed; only the placeholder content rendered today will change once the LangGraph executor is implemented.

Directory contents

File	Purpose
00_STATUS.json	Machine-readable summary of the run (packet id, status, timestamps).
01_MORNING_BRIEF.md	Top-line synthesis: what matters today, in priority order.
02_TODAY_PRIORITIES.md	Ordered list of priorities with rationale and source citations.
03_ACTIONS_BY_PERSON.md	Grouped by stakeholder: what each owns and what you owe each.
04_DECISIONS_NEEDED.md	Items requiring your call before they can move.
05_RISKS_AND_BLOCKERS.md	Flagged risks, with criticality and recommended next action.
06_DRAFT_MESSAGES.md	Drafts of messages you might want to send. Never sent automatically.
07_MEETING_PREP.md	Per-meeting prep notes: who, agenda, what to push, what to listen for.
08_CLOUD_REVIEW_CANDIDATES.md	Lists the local model flagged as candidates for cloud review (only used if cloud policy per
09_AUDIT_LOG.md	What ran, what models were called, what the evaluator said.

Sample 01_MORNING_BRIEF.md (mock content)

```
# Morning Brief &mdash; 2026-05-17

## Today's three things

1. Decide Q3 staffing for the Scope1 project. Three weeks of notes point at a hire vs reallocate question. Source: 04_DECISIONS_NEEDED.md.
2. Reply to the legal followup on the data sharing agreement. Draft in 06_DRAFT_MESSAGES.md ready for your review.
3. 30-min prep for the board sync at 14:00. Agenda + likely Qs in 07_MEETING_PREP.md.

## Risks surfaced

- Vendor X has not confirmed renewal terms (deadline Friday).
- The migration runbook from 2026-05-10 still has two unowned steps.

## Sources used

- ~/LocalAI/inbox/notes/2026-W20-recap.md
```

- ~/LocalAI/inbox/transcripts/2026-05-15-scope1-1on1.md
- ~/LocalAI/inbox/docs/2026-Q3-staffing-draft.md

Confidence

Medium-high. Six assumptions labeled in 02_TODAY_PRIORITIES.md.

Memory candidates extracted from this run land in

~/Obsidian/LocalAI-ChiefOfStaff/00_Inbox/Memory_Review/ for your approval before they enter long-term memory.

16. Configuration recipes

Three schedule profiles you might pick from.

Recipe: "strict night worker" (default)

```
day_mode_start: '07:30'  
night_mode_start: '01:30'  
night_mode_end: '07:00'
```

Day window is long (07:30 - 01:30 next day). Night execution starts late and ends well before you wake up. Good for the typical knowledge-worker schedule. Heavy models run with the laptop on charger overnight; you wake up to the brief.

Recipe: "early bird"

```
day_mode_start: '05:30'  
night_mode_start: '23:00'  
night_mode_end: '05:00'
```

Night starts at 23:00, leaving six hours of model time before the day window opens at 05:30. Suits someone who's at their desk by 06:00 reading briefs.

Recipe: "weekday office, weekend lab"

```
day_mode_start: '08:00'  
night_mode_start: '20:00'  
night_mode_end: '07:30'
```

Longer night window (11.5 hours) for someone who runs experiments or larger batch work overnight. The day window is shorter, so models stay safely untouched during working hours unless you explicitly day-unlock.

Threshold tuning

```
# Stricter: require near-perfect packets to go ready  
minimum_readiness_for_overnight: 0.92  
minimum_readiness_for_high_stakes: 0.95  
  
# Looser: accept more borderline packets (NOT recommended)  
minimum_readiness_for_overnight: 0.75  
minimum_readiness_for_high_stakes: 0.85
```

Lowering thresholds is rarely the right answer. Below 0.85, blocking gaps are almost guaranteed and the system produces lower-quality output that you'll have to redo. Stricter is fine; looser is a tax on tomorrow's you.

17. Model selection by hardware

Pick tags appropriate for your VRAM. The default config assumes a beefy M2 Max with 77+ GiB Metal VRAM. Smaller machines need smaller tags.

Hardware	local-main	local-secondary	local-coder	local-reasoner
96 GB unified (M2/M3 Max)	qwen3:30b-a3b	llama3.3:70b	qwen3-coder:30b	deepseek-r1:8b
64 GB unified	qwen3:30b-a3b	(skip 70B)	qwen3-coder:30b	deepseek-r1:8b
32 GB unified	qwen3:14b	qwen3:32b	qwen2.5-coder:14b	deepseek-r1:8b
16 GB unified	qwen3:8b	qwen3:14b	qwen2.5-coder:7b	deepseek-r1:8b
Linux NVIDIA 24GB	qwen3:30b-a3b	qwen3:32b	qwen3-coder:30b	deepseek-r1:8b

When you change tags

Both config/models.yaml and config/litellm.yaml must reflect the same tag for each group. The Models tab in the panel will pull the name from models.yaml for the gate; LiteLLM will use the litellm.yaml entry for routing.

```
# config/models.yaml
local-main: qwen3:30b-a3b      # change this

# config/litellm.yaml
- model_name: local-main
  litellm_params:
    model: ollama/qwen3:30b-a3b # and this, in lock-step
    api_base: http://localhost:11434
```

Why MoE 30B-a3b is the workhorse default

Qwen3's 30B Mixture-of-Experts model with 3B active parameters gives 30B-class quality at roughly 8B-class inference speed. Excellent fit for batch overnight work on Apple Silicon: weights load quickly, per-token latency is low, instruction following is strong for clarification-style tasks.

18. What's implemented vs scaffolded

Capability	State	Notes
Deterministic clarification flow	Implemented	Builder, 9-dim readiness, question gen. 100% test-covered.
Postgres storage layer	Implemented	7 tables, schema migrated, CRUD for packets/questions/runs.
Safety gates (5 functions)	Implemented	Parameterized tests over every local-* group and mode.
Day-unlock switch	Implemented	File + env channels, visible banner, fail-closed.
Streamlit control panel	Implemented	Header strip, 7 tabs, gate-aware UI.
Models tab (load/unload/test)	Implemented	CLI workflow surfaced: Load -> Quick test -> Unload.
Schedule tab (time editing)	Implemented	Atomic config writes, validation, launchd-status mirror.
Ollama integration	Implemented	Pull, list, load, unload, generate.
Temporal worker + workflow defs	Skeleton	Workflows + activities defined; bodies return placeholders.
LiteLLM routing	Config only	Started via start_litellm.sh. Used by future call sites.
Launchd plist generation	Implemented	Reads night_mode_start; arming stays manual.
Overnight execution graph (top-level steps)	Implemented	11-step pipeline in execution/graph.py iterated by the thin runner.
File classifier -> model routing	Implemented	Code suffixes -> local-coder; everything else -> local-main.
Secondary local retry chain	Implemented	Primary fails evaluator -> retry on local-secondary; promotes retry.
Cloud-review gate (catalog mode)	Implemented	Files where both local attempts fail are catalogued in 08_CLOUD.
Claude cloud-review caller (actual HTTP)	Scaffolded	Gate runs and catalogs candidates; the actual HTTP call to Claude.
Self-consistency / self-critique loops	Scaffolded	Extraction is single-shot today. N-of-3 voting and self-critique are.
Pause/resume UI	Scaffolded	Buttons disabled. DB statuses exist.
Semantic memory indexing	Scaffolded	Chroma client exists. No indexer yet.
Drafted messages	Scaffolded	Output template exists; nothing fills it yet.
PDF documentation builder	Implemented	scripts/build_pdfs.py (thin entry) + scripts/pdf_builders/ package.

19. File system layout

Project repository

```
local-ai-orchestrator/
  AGENTS.md          rules for human/AI editors
  ONBOARDING.md      practical usage guide
  README.md          setup + ports + safety summary
  app/control_panel.py Streamlit UI
  assistant_core/    Python package
    clarification/    work packet builder, readiness, questions
    execution/        18-step pipeline (scaffolded)
    llm/              ollama_admin, litellm_client, model_policy
    memory/           Obsidian + Chroma writers
    temporal_app/     worker, workflows, activities
    safety.py         5 gate functions
    schemas.py        pydantic models for the whole system
    storage/          Postgres CRUD package (by-entity submodules)
    config.py         yaml config loader
    config_writer.py  atomic config edits
    scheduler_status.py launchd + service health
  config/
    assistant.yaml    day/night times, thresholds, paths
    models.yaml        group -> Ollama tag mapping
    memory.yaml        vector store settings
    litellm.yaml       LiteLLM router config (gitignored)
  db/migrations/      SQL applied on Postgres first boot
  docs/               this PDF and friends, plus design docs
  examples/           sample answers, sample packet
  launchd/            generated plist (gitignored)
  scripts/            bash entrypoints for every operation
  tests/              pytest, isolated from real state
  docker-compose.yml  postgres + temporal + temporal-ui + open-webui
```

Data directories (outside the repo)

```
~/LocalAI/
  inbox/
    notes/           your input notes
    transcripts/      call/meeting transcripts
    docs/             documents
    csv/              tabular sources
    cloud_review/     files marked for cloud review (Claude gate path)
  output/<date>/      generated artifacts (briefs, priorities, etc.)
  archive/            processed inputs
  logs/              runtime logs (incl. launchd stdout/stderr)
  state/              operational state (incl. day_unlock.flag)
  work_packets/       packet-specific files

~/Obsidian/LocalAI-ChiefOfStaff/
  00_Inbox/Memory_Review/ memory candidates awaiting approval
  01_Daily_Briefs/        morning briefs
  02_Work_Packets/        packet notes
```

03_Projects/	project pages
04_Meetings/	meeting notes
05_Decisions/	logged decisions
06_Stakeholders/	people pages
07_Playbooks/	reusable procedures
08_Prompts/	curated prompts
09_System_Logs_Summaries/	run summaries
99_Archive/	historical

20. Database schema

Seven tables in the localai database, all created from db/migrations/001_init.sql:

Table	Purpose
work_packets	One row per packet: title, objective, status, readiness, policies (jsonb), raw and structured request
clarification_questions	Per-packet questions across rounds; each can be answered or unanswered.
execution_runs	One row per overnight invocation, with Temporal workflow id, mode, start/end timestamps.
model_calls	Audit row per local-model or cloud call: task type, group, actual tag, sizes, success/error.
evaluations	Quality / grounding / completeness / contradiction scores from the evaluator, with recommended
artifacts	File-level pointers to generated outputs and their Obsidian copies.
memory_candidates	Items extracted from outputs as candidates for long-term memory; require user approval.

Looking at the data directly

```
# Open a psql shell against the local Postgres container
docker exec -it localai-postgres psql -U localai -d localai

# In psql:
\dt                                -- list tables
SELECT id, title, status, readiness_score, created_at
  FROM work_packets ORDER BY created_at DESC LIMIT 5;

SELECT round_number, category, question, answered
  FROM clarification_questions
 WHERE work_packet_id = 'YOUR-UUID'
 ORDER BY round_number, priority DESC;
```

All tables use UUID primary keys. JSONB columns on work_packets allow policy evolution without migrations. Foreign keys enforce relational integrity.

21. Code roadmap (file-by-file)

Where to look when you want to change something.

When you want to…	Look here
change clarification question wording	assistant_core/clarification/question_generator.py
change readiness weights or thresholds	assistant_core/clarification/readiness.py + config/assistant.yaml
change which fields the work packet has	assistant_core/schemas.py
change what "high stakes" detects	assistant_core/clarification/work_packet_builder.py
add a safety gate or tighten an existing one	assistant_core/safety.py
add a new model group	assistant_core/safety.py (LOCAL_MODEL_GROUPS) + config/models.yaml + co
change the day-unlock UI behavior	app/control_panel.py (_render_header) + scripts/day_unlock.sh
wire the actual overnight execution pipeline	assistant_core/execution/graph.py + assistant_core/temporal_app/activities.py
add a Postgres column	db/migrations/ (add a new file) + assistant_core/storage/
change LiteLLM routing	config/litellm.yaml + assistant_core/llm/litellm_client.py
add a new tab to the panel	app/control_panel.py (new _render_xxx_tab function + register)
add a new background service status	assistant_core/scheduler_status.py
change what files Ollama admin lists	assistant_core/llm/ollama_admin.py
add a new editable config field	assistant_core/config_writer.py (EDITABLE_FIELDS) + config/assistant.yaml
change the launchd schedule template	scripts/create_launchd_plist.sh
regenerate these PDFs	scripts/build_pdfs.py

22. Anti-patterns and pitfalls

Mistakes that are easy to make. Each one is real (some are things I, your co-author, almost made you do at some point):

Loading both 30B models simultaneously to "test things"

Two 30B-class models in VRAM is 60+ GB on M2 Max. The OS starts shuffling, the first inference takes 30+ seconds, and you'll blame the orchestrator. Unload one before testing the other.

Lowering readiness thresholds to "get things moving"

Tempting when a packet just won't pass. Almost always wrong: the blocking gaps are blocking *because* the system can't safely produce useful output without them. Lower the bar and you get a confidently wrong morning brief.

Leaving day_unlock on overnight

The flag persists across reboots. The Streamlit banner makes it visible, but the banner is only loud if you're *looking* at the panel. Make day_lock.sh part of your end-of-day habit.

Pointing the orchestrator at ~/Documents

~/LocalAI/inbox/ is the source directory by convention. Pointing the system at broader user data invites accidents: a tag indexer might pick up personal docs, an evaluator might leak content into outputs you share. Stay in the sandbox.

Setting ANTHROPIC_API_KEY "just in case"

Setting the key satisfies one of the six cloud gates. If you also set cloud_fallback_enabled: true in config and one of your packets happens to mark cloud allowed, a high-stakes failure path could escalate. Don't pre-arm a gun you don't plan to fire.

Skipping Temporal because it's "just clarification"

True today — clarification is synchronous Python and doesn't touch Temporal. But the moment execution is wired, the worker needs to be running or workflows queue and never run. Bring up the worker as part of the start sequence so you don't get surprised at 06:00.

Editing files in ~/LocalAI/output/ expecting them to persist

Outputs are regenerated on each run; the dated directory is overwritten. Anything you want to keep, copy to your Obsidian vault. The vault is the durable layer.

Running `stop\_services.sh` and assuming everything is off

That script only stops Docker containers. Streamlit, Ollama, LiteLLM, and the Temporal worker are host processes. They survive. If you want everything down: stop_services.sh + Ctrl+C the Streamlit tab

+ brew services stop ollama + Ctrl+C the worker.

23. Glossary

Term	Definition
Work packet	The orchestrator's unit of work. A structured representation of a request with objective, sources,
Readiness score	Weighted sum of nine binary dimension checks, in [0, 1]. Threshold 0.85 to be eligible for overnight
Clarification round	One pass of: build packet -> score -> generate questions -> wait for answers.
DAY_MODE	Mode in which all local-* model groups are blocked at the safety gate unless day_unlock is active
NIGHT_MODE	Mode in which local model calls are allowed. Required by the overnight runner.
MANUAL_RESUME	Third mode value. Allows the overnight runner to proceed when you're explicitly re-running pause
Day unlock	An explicit, visible override that lets local models load in DAY_MODE. Implemented as a flag file
Model group	A logical role (local-main, local-coder, etc.) that maps to a specific Ollama tag in config/models.y
Keep-alive	Ollama's per-model VRAM retention timer. Our Load uses 30m; Quick test uses 5m; Unload sen
Cloud-review path	The Claude Opus route, gated by six policy checks. Files must be in ~/LocalAI/inbox/cloud_revie
Auto-execution	The launchd job that runs run_ready_overnight.sh at night_mode_start. Off by default; arming is
Manual override	A per-call boolean (manual_override=True) that bypasses the day gate. Used in tests and direct
Sentinel file	A small file whose existence (not contents) signals a state. day_unlock.flag is one; the launchd p
Fail-closed	Default-to-block when input is ambiguous, missing, or unknown. Opposite of fail-open. The right
Memory candidate	A statement extracted from a run that might be worth keeping long-term. Lands in the Obsidian m
MoE	Mixture of Experts. Models like qwen3:30b-a3b have 30B parameters but only ~3B active per tok

End of System Guide. For practical usage recipes and day-to-day operation, see the companion Onboarding PDF.